



CS4001NI Programming
60% Individual Coursework

2025 Spring

Student Name: Shyagi Kunwar

London Met ID: 24046505

College ID: NP01NT4A240176

Group: N2

Assignment Due Date: Friday, May 16, 2025

Assignment Submission Date: Friday, May 16, 2025

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a marks of zero will be awarded.

4% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- 
9 Not Cited or Quoted 3%
 Matches with neither in-text citation nor quotation marks
- 
0 Missing Quotations 0%
 Matches that are still very similar to source material
- 
3 Missing Citation 1%
 Matches that have quotation marks, but no in-text citation
- 
0 Cited and Quoted 0%
 Matches with in-text citation present, but no quotation marks

Top Sources

- 0%  Internet sources
- 0%  Publications
- 4%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Table of Contents

1. Introduction	1
2. GUI Wireframe	2
3. Class Diagram	5
4. Pseudocode	6
5. Method Description	21
6. Testing	22
6.1 Test – 1:	22
6.2 Test – 2:	24
7. Error Detection and Correction	26
7.1 Syntax Error	26
7.2 Semantic Error	27
7.3 Logical Error	28
8. Conclusion	29
9. Appendix	30

List of Figures

Figure 1: Main Menu Wireframe	2
Figure 2: Regular Member Wireframe	3
Figure 3: Premium Member Wireframe	4
Figure 4: Class Diagram.....	5
Figure 5: Add Regular Member	23
Figure 6: Add Premium Member	23
Figure 7: Marked Attendance Regular Member	25
Figure 8: Marked Attendance Premium Member.....	25

List of Tables

Table 1: Test - 1 22

Table 2: Test - 2 24

1. Introduction

This coursework is assigned to us students for the reason of skill and knowledge development on Java programming using BlueJ. This coursework is designed by the teachers to introduce Java programming between the students studying related subjects. Java is one of the most used programming languages throughout the world and it is important that a studying an IT related should know about the basics of Java programming. Through this given coursework, students can develop a strong base regarding programming and has also introduced the principle and concepts of Java programming, which is the principle of OOP, classes, object, inheritance, polymorphism and other. This will also help the students to build a strong foundation, and they will be able to face more complex Java programming.

The main aim of this coursework is to introduce the new concept of Java programming between the new beginner students studying in first year IT, so the student can learn how to write at least basics of Java coding. At the end of this project students will have a strong build, knowledge and skill about Java programming within BlueJ. BlueJ is a very good platform for learning the basics and bases of Java programming. One of the aims of this coursework is to familiarize the students with the Java programming environment through BlueJ. In today's world coding plays a very vital role because every application needs a specific coding, without coding the online world seems a little impossible to imagine. The focus of this coursework is to encourage best coding practises within the students at the college.

2. GUI Wireframe

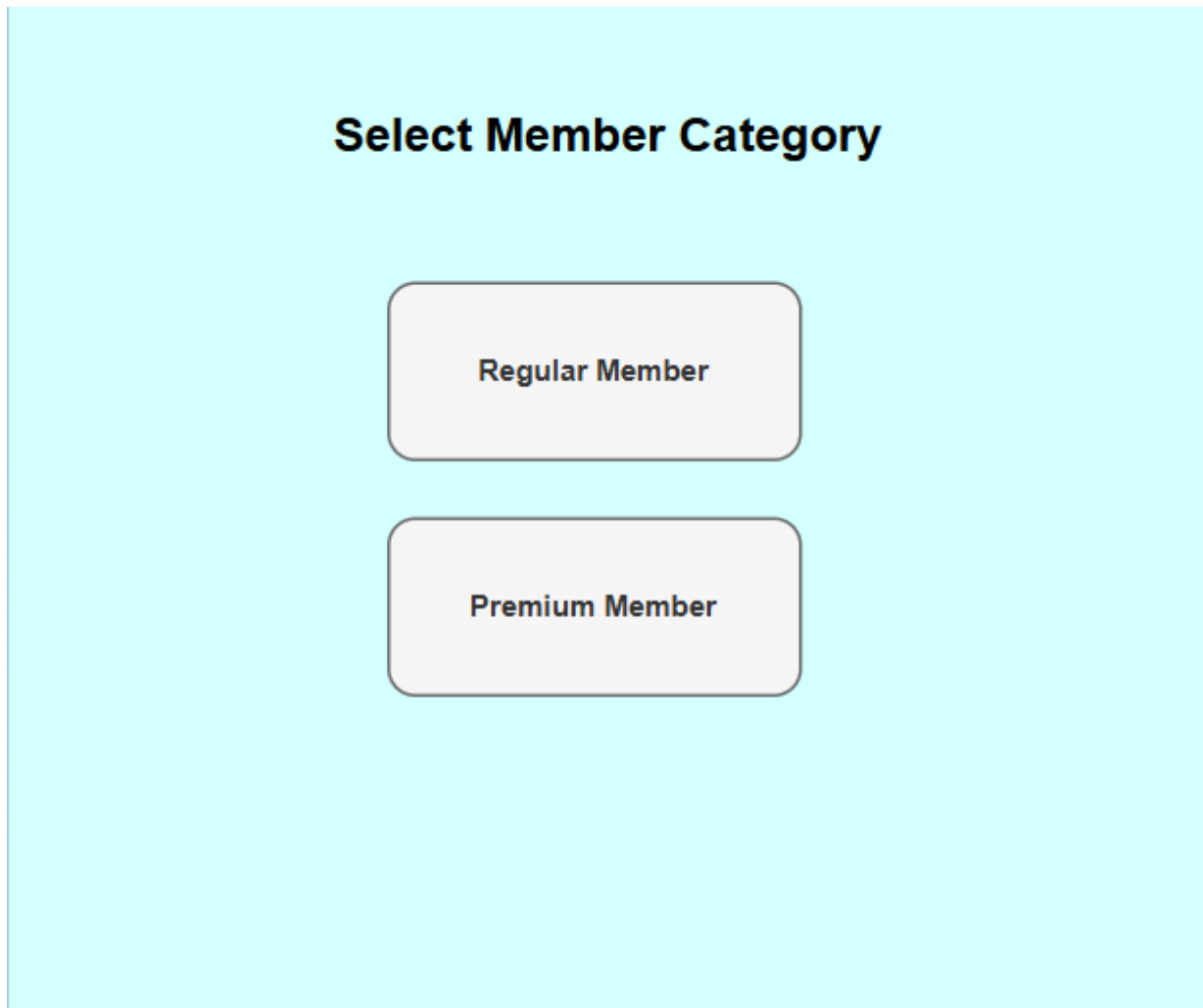


Figure 1: Main Menu Wireframe

Regular Member Entry

ID		Gender	Select	
Name				Activate
Phone		Plan	Select	Deactivate
Email				Add Regular Member
Address		Payment Method	Select	
Referred By		ID		Revert Regular Member
Paid Amount		ID		
Removal Reason		ID		
Date of Birth		ID		

Figure 2: Regular Member Wireframe

Premium Member Entry

ID		Gender	Select	
Name				Activate
Phone		Plan	Select	Deactivate
Email				Add Premium Member
Address		Payment Method	Select	Revert Premium Member
Referred By		ID		
Paid Amount		ID		
Removal Reason		ID		
Date of Birth		ID		

Figure 3: Premium Member Wireframe

3. Class Diagram

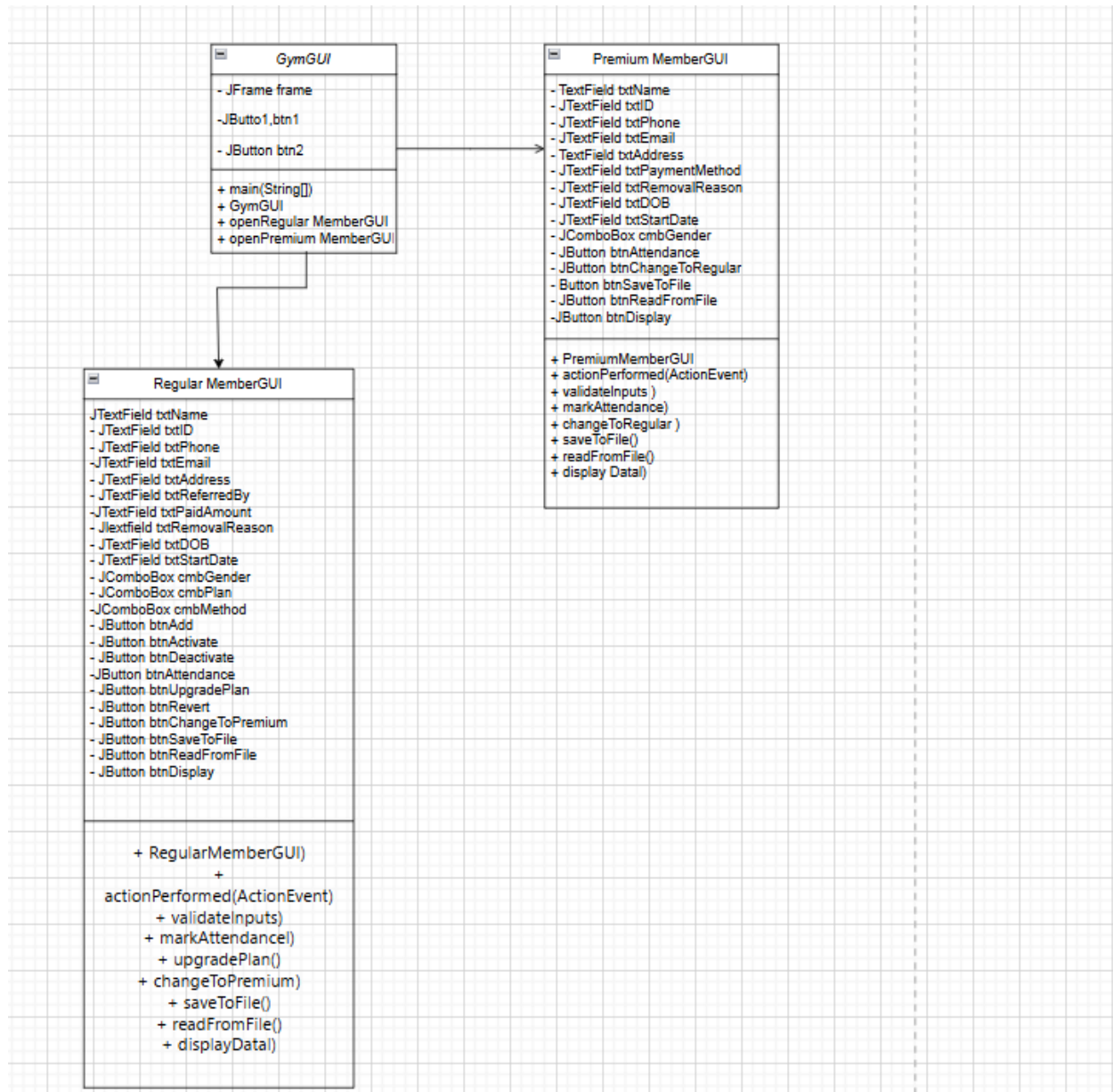


Figure 4: Class Diagram

4. Pseudocode

```
// Class GymGUI
CLASS GymGUI
    // Instance variable to hold attendance log (ArrayList of some type, details not
    specified)
    attendanceLog: ArrayList

    // Main method (entry point of the program)
    METHOD main (args: Array of String)
        CREATE new GymGUI object
        CALL createMainMenu on the new object
    END METHOD

    // Method to create the main menu GUI
    METHOD createMainMenu()
        CREATE JFrame named menu with title "Gym Membership"
        SET menu size to 800x600
        SET menu resizable to true
        SET default close operation of menu to EXIT
        SET menu visibility to true
        SET layout of menu to null (absolute positioning)
        SET menu to maximized state
        SET background color of menu content pane to Alice Blue

        CREATE JLabel named title with text "Select Member Category"
        SET font of title to Arial, Bold, size 24
        SET bounds of title to (500, 30, 400, 40)
```

CREATE JButton named regBtn with text "Regular Member"

SET bounds of regBtn too (520, 120, 240, 60)

ADD ActionListener to regBtn:

ON actionPerformed:

CREATE new RegularMemberGUI object

SET layout of regBtn to BorderLayout

CREATE JButton named premBtn with text "Premium Member"

SET bounds of premBtn to (520, 210, 240, 60)

ADD ActionListener to premBtn:

ON actionPerformed:

CREATE new PremiumMemberGUI object

SET layout of premBtn to BorderLayout

ADD title to menu

ADD regBtn to menu

ADD premBtn to menu

SET menu visibility to true

END METHOD

END CLASS

// Class RegularMemberGUI

CLASS RegularMemberGUI

// Instance variables for various input fields (JTextField, JComboBox, JTextArea)

id: JTextField

name: JTextField

phone: JTextField

email: JTextField

address: JTextField

referredby: JTextField

```
paidAmt: JTextField
removalreason: JTextField
dob: JTextField
gender: JComboBox
method: JComboBox
plans: JComboBox
displayArea: JTextArea
id1: JTextField
id2: JTextField
id3: JTextField
id4: JTextField
```

```
// Constructor for RegularMemberGUI
```

```
CONSTRUCTOR RegularMemberGUI ()
```

```
    CREATE JFrame named win with title "Regular Member"
```

```
    SET win to maximized state
```

```
    SET layout of win to null
```

```
    SET background color of win content pane to Alice Blue
```

```
    CREATE JLabel named lbl with text "Regular Member Entry"
```

```
    SET bounds of lbl to (500, 10, 300, 30)
```

```
    SET font of lbl to Arial, Bold, size 20
```

```
    ADD lbl to win
```

```
    CREATE String array genders with values {"Select", "Male", "Female", "Other"}
```

```
    CREATE String array planList with values "Basic", "Regular", "Premium"
```

```
    CREATE String array methods with values "Cash", "Card", "Online"
```

```
CALL createField with label "ID", y-coordinate 90, and win to create id JTextField
```

CALL createField with label "Name", y-coordinate 130, and win to create name
JTextField

CALL createField with label "Phone", y-coordinate 170, and win to create phone
JTextField

CALL createField with label "Email", y-coordinate 210, and win to create email
JTextField

CALL createField with label "Address", y-coordinate 250, and win to create address
JTextField

CALL createField with label "Referred By", y-coordinate 290, and win to create
referredby JTextField

CALL createField with label "Paid Amount", y-coordinate 330, and win to create
paidAmt JTextField

CALL createField with label "Removal Reason", y-coordinate 370, and win to create
removalreason JTextField

CALL createField with label "Date of Birth", y-coordinate 410, and win to create dob
JTextField

CREATE JComboBox gender with genders array

SET bounds of gender too (600, 90, 150, 25)

ADD JLabel with text "Gender" and bounds (500, 90, 80, 25) to win

ADD gender to win

CREATE JComboBox plans with planList array

SET bounds of plans too (600, 130, 150, 25)

ADD JLabel with text "Plan" and bounds (500, 130, 80, 25) to win

ADD plans to win

CREATE JComboBox method with methods array

SET bounds of method to (600, 170, 150, 25)

ADD JLabel with text "Pay Method" and bounds (500, 170, 80, 25) to win

ADD method to win

```
// Create and add labels and text fields for ID 1 to ID 4
CREATE JLabel idLabel1 with text "ID"
SET bounds of idLabel1 to (500, 240, 100, 25)
ADD idLabel1 to win
CREATE JTextField idField1
SET bounds of idField1 to (600, 240, 150, 25)
ADD idField1 to win

// Similar blocks for idLabel2, idField2, idLabel3, idField3, idLabel4, idField4

CREATE JButton activate with text "Activate"
SET bounds of activate to (900, 80, 160, 30)
ADD ActionListener to activate:
  ON actionPerformed:
    GET text from id JTextField
    IF text is empty:
      SHOW message dialog on win with message "Enter member ID to activate."
    ELSE:
      SHOW message dialog on win with message "Member " + text + " activated." //
Placeholder for actual logic
ADD activates to win

CREATE JButton deactivate with text "Deactivate"
SET bounds of deactivate to (900, 140, 160, 30)
ADD ActionListener to deactivate:
  ON actionPerformed:
    GET text from id JTextField
    IF text is empty:
      SHOW message dialog on win with message "Enter member ID to deactivate."
    ELSE:
```

SHOW message dialog on win with message "Member " + text + " deactivated." // Placeholder for actual logic

ADD deactivates to win

CREATE JButton addRegular with text "Add Regular Member"

SET bounds of addRegular to (900, 200, 160, 30)

ADD ActionListener to addRegular:

ON actionPerformed:

GET text from name JTextField

GET text from id JTextField

IF name text is empty OR id text is empty:

SHOW message dialog on win with message "Enter member ID and name to add."

ELSE:

SHOW message dialog on win with message "Regular member " + name text + " added." // Placeholder for actual logic

ADD addRegular to win

CREATE JButton revertRegular with text "Revert Regular Member"

SET bounds of revertRegular to (875, 260, 210, 40)

ADD ActionListener to revertRegular:

ON actionPerformed:

GET text from id JTextField

IF text is empty:

SHOW message dialog on win with message "Enter member ID to revert."

ELSE:

SHOW message dialog on win with message "Member " + text + " reverted to previous status." // Placeholder for actual logic

ADD revertRegular to win

CREATE JButton mark with text "Mark Attendance"

```
CREATE JButton upgrade with text "Upgrade Plan"
CREATE JButton toPremium with text "Change to Premium"
CREATE JButton save with text "Save"
CREATE JButton clear with text "Clear"
CREATE JButton display with text "Display"

// Set bounds for the buttons
SET bounds of mark to (100, 530, 160, 30)
SET bounds of upgrade to (280, 530, 160, 30)
SET bounds of toPremium to (460, 530, 180, 30)
SET bounds of save too (660, 530, 160, 30)
SET bounds of display to (840, 530, 160, 30)
SET bounds of clear to (1020, 530, 160, 30)

// Add action listeners to the buttons
ADD ActionListener to mark:
    ON actionPerformed:
        SHOW message dialog on win with message "Attendance Marked: " + text of
name JTextField
ADD ActionListener to upgrade:
    ON actionPerformed:
        SHOW message dialog on win with message "Plan Upgraded to: " + selected item
of plans JComboBox
ADD ActionListener to toPremium:
    ON actionPerformed:
        SHOW message dialog on win with message "Switched to Premium Membership"
ADD ActionListener to save:
    ON actionPerformed:
        CALL saveToFile
ADD ActionListener to clear:
    ON actionPerformed:
```

```
        CALL clearFields
    ADD ActionListener to display:
        ON actionPerformed:
            CALL displayInfo

// Add buttons to win
ADD mark to win
ADD upgrade to win
ADD toPremium to win
ADD save to win
ADD display to win
ADD clear to win

SET win visibility to true
END CONSTRUCTOR

// Helper method to create a labeled text field
METHOD createField(label: String, y: Integer, frame: JFrame): JTextField
    CREATE JLabel named lbl with text label
    SET bounds of lbl to (50, y, 100, 25)
    CREATE JTextField named tf
    SET bounds of tf to (150, y, 200, 25)
    ADD lbl to frame
    ADD tf to frame
    RETURN tf
END METHOD

// Method to save data to a file
METHOD saveToFile()
    TRY
        CREATE PrintWriter named pw for file "regular.txt"
```

```
    WRITE to pw: text of id JTextField + "," + text of name JTextField
    SHOW message dialog with message "Saved to File"
    CLOSE pw
    CATCH IOException ex
        SHOW message dialog with message "Error Saving"
    END TRY
END METHOD
```

```
// Method to clear all input fields
METHOD clearFields()
    SET text of id JTextField to ""
    SET text of name JTextField to ""
    SET text of phone JTextField to ""
    SET text of email JTextField to ""
    SET text of address JTextField to ""
    SET text of referredby JTextField to ""
    SET text of paidAmt JTextField to ""
    SET text of removalreason JTextField to ""
    SET text of dob JTextField to ""
    SET text of id1 JTextField to ""
    SET text of id2 JTextField to ""
    SET text of id3 JTextField to ""
    SET text of id4 JTextField to ""
    SET text of displayArea JTextArea to ""
END METHOD
```

```
// Method to display member information
METHOD displayInfo()
    SET text of displayArea JTextArea to:
```

```
"Name: " + text of name JTextField + "\n" +  
"Phone: " + text of phone JTextField  
END METHOD  
END CLASS  
  
// Class RegularMember (appears to be a different approach/version of Regular  
Member UI)  
CLASS RegularMember  
    // Instance variables for GUI components  
    frame: JFrame  
    idField: JTextField  
    nameField: JTextField  
    phoneField: JTextField  
    emailField: JTextField  
    addressField: JTextField  
    referralField: JTextField  
    dobField: JTextField  
    startDateField: JTextField  
    removalField: JTextField  
    planBox: JComboBox  
    genderBox: JComboBox  
    displayArea: JTextArea  
  
    // Constructor  
    CONSTRUCTOR RegularMember ()  
        CREATE JFrame named frame with title "Regular Member"  
        SET frame size to 600x600  
        SET layout of frame to BorderLayout  
  
        CREATE JPanel named formPanel with GridBagLayout  
        CREATE GridBagConstraints named gbc
```

SET gbc insets, fill, and anchor

// Add labels and text fields to formPanel using GridBagConstraints

// (Similar pattern for ID, Name, Phone, Email, Address, Referred By, Plan, Gender, DOB, Start Date, Removal Reason)

ADD formPanel to frame with BorderLayout.NORTH

CREATE JTextArea named displayArea with 8 rows and 50 columns

SET displayArea editable to false

CREATE JScrollPane with displayArea

ADD scrollPane to frame with BorderLayout.CENTER

CREATE JPanel named buttonPanel with GridLayout (1 row, 6 columns, horizontal and vertical gaps)

CREATE JButtons (saveButton, readButton, displayButton, upgradeButton, markButton, changeToPremium)

// Add buttons to buttonPanel

ADD buttons to buttonPanel

ADD buttonPanel to frame with BorderLayout.SOUTH

// Add ActionListeners to buttons (saveToFile, readFromFile, display, etc.)

// ... (Logic for each button's action)

SET frame to maximized state

SET default close operation of frame to EXIT

SET frame visibility to true

END CONSTRUCTOR

```
// Method to save data to file "regular_member.txt"
METHOD saveToFile()
    // ... (Write data from text fields and combo boxes to the file)
END METHOD

// Method to read data from file "regular_member.txt"
METHOD readFromFile()
    // ... (Read data from the file and display it in displayArea)
END METHOD

// Method to display member information in displayArea
METHOD display()
    // ... (Format and set text of displayArea with data from input fields)
END METHOD

// Main method for testing (runs this RegularMember UI independently)
METHOD main(args: Array of String)
    CALL SwingUtilities.invokeLater with lambda: CREATE new RegularMember object
END METHOD
END CLASS

// Class PremiumMemberGUI
CLASS PremiumMemberGUI
    // Instance variables for input fields (similar to RegularMemberGUI)
    id: JTextField
    name: JTextField
    phone: JTextField
    email: JTextField
    address: JTextField
    trainer: JTextField
```

```
payment: JTextField
removalreason: JTextField
dob: JTextField
gender: JComboBox
method: JComboBox
plans: JComboBox
view: JTextArea // Note: 'view' seems unused in the provided code
displayArea: JTextArea
id1: JTextField
id2: JTextField
id3: JTextField
id4: JTextField
referredby: JTextField
```

```
// Constructor
```

```
CONSTRUCTOR PremiumMemberGUI()
```

```
    // ... (Similar GUI setup as RegularMemberGUI, but with fields specific to Premium
members like trainer and payment)
```

```
    // ... (Includes labels, text fields, combo boxes, buttons for Activate, Deactivate, Add
Premium, Revert Premium, Mark Attendance, Upgrade Plan, Change to Regular, Save,
Clear, Display)
```

```
    // ... (ActionListeners for buttons with similar logic as RegularMemberGUI, adapted for
Premium members)
```

```
END CONSTRUCTOR
```

```
// Helper method to create labeled text field (same as RegularMemberGUI)
```

```
METHOD createField(label: String, y: Integer, frame: JFrame): JTextField
```

```
    // ...
```

```
END METHOD
```

```
// Method to save premium member data to "premium.txt"
```

```
METHOD saveToFile()
```

```
    // ... (Write data from premium member fields to the file)
```

```
END METHOD
```

```
// Method to clear premium member input fields
```

```
METHOD clear()
```

```
    // ... (Clear all text fields and the display area)
```

```
END METHOD
```

```
// Method to display premium member information
```

```
METHOD display()
```

```
    // ... (Set text of view JTextArea with name and phone)
```

```
END METHOD
```

```
END CLASS
```

```
// Class PremiumMember (another approach/version of Premium Member UI)
```

```
CLASS PremiumMember
```

```
    // Instance variables for GUI components
```

```
    frame: JFrame
```

```
    idField: JTextField
```

```
    nameField: JTextField
```

```
    phoneField: JTextField
```

```
    emailField: JTextField
```

```
    addressField: JTextField
```

```
    loyaltyField: JTextField
```

```
    paidField: JTextField
```

```
    dobField: JTextField
```

```
    startDateField: JTextField
```

```
    removalField: JTextField
```

```
    planBox: JComboBox
```

```
    genderBox: JComboBox
```

displayArea: JTextArea

// Constructor

CONSTRUCTOR PremiumMember()

CREATE JFrame named frame with title "Premium Member"

SET frame to maximized state

SET layout of frame to FlowLayout

// Add labels and text fields for Premium member attributes

// (ID, Name, Phone, Email, Address, Loyalty Points, Plan, Gender, DOB, Start Date, Removal Reason)

CREATE JButtons (saveButton, readButton, displayButton, downgradeButton, markButton, switchToRegular)

SET preferred sizes for buttons

CREATE JTextArea displayArea with scroll pane

ADD displayArea and buttons to frame

// Add ActionListeners to buttons (saveToFile, readFromFile, display, etc.)

// ... (Logic for each button's action)

SET frame visibility to true

END CONSTRUCTOR

// Method to save premium member data to "premium_member.txt"

METHOD saveToFile()

// ... (Write data to file)

END METHOD

// Method to read premium member data from "premium_member.txt"

```
METHOD readFromFile()  
    // ... (Read data from file and display)  
END METHOD  
  
// Method to display premium member information  
METHOD display()  
    // ... (Format and display data)  
END METHOD  
END CLASS
```

5. Method Description

- `addMember ()` – This method collects all the information from the textField (ID, name, Email, Address, DOB, etc.) and then adds all this information for the new person to the information entries for validation.
- `activateMembership ()` – This method is used to denote that the member is actively a gym member and that he has activated his membership. You can find whether a member is active or not through their ID or name.
- `deactivateMembership ()` - This method is used to denote that the member is not actively a gym member and that he has deactivated his membership. You can find whether a member is deactivated or not through their ID or name.
- `markAttendance ()` – This shows the recorded attendance of a member, it can be accessed through ID or name.

6. Testing

6.1 Test – 1:

Objective	To add members in regular and premium
Action	All the required fields were filled and add premium customer button was clicked
Expected Output	A dialog box with a message “Regular member added successfully” “Premium member added successfully” should be shown.
Actual Output	“Regular member added successfully” “Premium member added successfully” was shown as expected
Result	Successful

Table 1: Test - 1

Regular member Entry

ID	7642374	Gender	Select	Activate
Name	Shyagi	Plan	Basic	Deactivate
Phone		Pay Method	Cash	Add Regular Member
Email				Revert Regular Member
Address		ID		
Referred By		ID		
Paid Amount		ID		
Removal Reason				
Date of Birth				

Message

Regular member Shyagi added.

OK

Figure 5: Add Regular Member

ID	742788562	Gender	Select	Activate
Name	Shyagi	Pay Method	Cash	Deactivate
Phone		Plan	Basic	
Email		ID		Add Premium Member
Address		ID		
Trainer		ID		
Amount				
Removal				
DOB				

Message

Premium member Shyagi added.

OK

Figure 6: Add Premium Member

6.2 Test – 2:

Objective	To mark attendance of Regular Member and Premium Member
Action	All the required fields were filled and add Regular Member and Premium Member button was clicked
Expected Output	A dialog box with a message “Attendance Marked” should be shown
Actual Output	A dialog box with a message “Attendance Marked” is shown as expected
Result	The result was Successful.

Table 2: Test - 2

The interface is divided into two main sections. The top section contains input fields for member details: ID, Name, Phone, Email, Address, Referred By, Paid Amount, Removal Reason, and Date of Birth. To the right of these fields are dropdown menus for Gender (Select), Plan (Basic), and Pay Method (Cash). Further right are buttons for Activate, Deactivate, Add Regular Member, and Revert Regular Member. The bottom section contains a row of buttons: Mark Attendance, Upgrade Plan, Change to Premium, Save, Display, and Clear. A modal message box is centered on the screen, displaying the text "Attendance Marked:" with an "OK" button.

Figure 7: Marked Attendance Regular Member

The interface is divided into two main sections. The top section contains input fields for member details: ID, Name, Phone, Email, Address, Trainer, Amount, Removal, and DOB. To the right of these fields are dropdown menus for Gender (Select), Pay Method (Cash), and Plan (Basic). Further right are buttons for Activate, Deactivate, and Add Premium Member. The bottom section contains a row of buttons: Mark Attendance, Upgrade Plan, Change to Regular, Save, Display, and Clear. A modal message box is centered on the screen, displaying the text "Attendance Mrked:" with an "OK" button.

Figure 8: Marked Attendance Premium Member

7. Error Detection and Correction

7.1 Syntax Error

A Syntax Error is a mistake which happens sometimes when there is no data type or method name is written, this can cause Syntax Error. You can understand it as of making a punctuation English.

```
// Private attributes  
private final double premiumCharge;  
private String personalTrainer;  
private isFullPayment;
```

Figure 9: Syntax Error

```
// Private attributes  
private final double premiumCharge;  
private String personalTrainer;  
private boolean isFullPayment;
```

Figure 10: Syntax Error Correction

7.2 Semantic Error

In this type of error where like Syntax error data type can be the issue, for example if an attribute has two different data types in one code it can show “incompatible error”.

```
public String getPremiumCharge()  
{  
    return this.premiumCharge;  
}
```

Figure 11: Semantic Error

```
public double getPremiumCharge()  
{  
    return this.premiumCharge;  
}
```

Figure 12: Semantics Error Correction

7.3 Logical Error

```
public void calculateDiscount()  
{  
    if (this.isFullPayment)  
    {  
        this.discountAmount = 0.1 * premiumCharge; // Calcul  
        System.out.println("Discount calculated: " + name);  
    }  
}
```

Figure 13: Logical Error

```
public void calculateDiscount()  
{  
    if (this.isFullPayment)  
    {  
        this.discountAmount = 0.1 * premiumCharge; // Calculate 10% d  
        System.out.println("Discount calculated: " + discountAmount);  
    }  
}
```

Figure 14: Logical Error Correction

8. Conclusion

In this given coursework by my college, we successfully learned to develop a Membership entry for a gym using Java (BlueJ) and modular programming practices. The system we created allows us to manage the members of the gym very Hassel freely by giving us the perfect data collecting and storing program. Through this given project we as students were able to learn the essentials basics of Java programming, from this GUI- based Gym Member Management System we learned Object-Oriented Programming, File handling, GUI (Graphical User Interface) designs, Problem solving, Program logic, code structure and real-world applications.

We also tested the program with different inputs to fully make sure it worked properly and gave all the data correctly. At the end the program runs smoothly and covers all the features and functions required, no error, run smoothly and User-friendly interactions Overall, this coursework helped enhance my understanding of fundamental programming concepts, improved my programming skills and gave me a practical experience in developing a first-hand Java system in the real-world. This project also helped us very much with what to expect when we are having a real-time experience the practical really gave us an insight into Java programming.

9. Appendix

```
import java.util.ArrayList;
import javax.swing.JFrame;
import javax.swing.JPanel;
import java.awt.Color;
import javax.swing.JButton;
import javax.swing.JComboBox;
import java.awt.Font;
import javax.swing.JTextField;
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.JOptionPane;
import java.io.PrintWriter;
import java.io.IOException;
import java.io.BufferedReader;
import java.io.FileReader;

public class GymGUI
{
    ArrayList<String> attendanceLog = new ArrayList<>();

    public static void main(String[] args)
    {
        new GymGUI().createMainMenu();
    }
}
```

```
}

public void createMainMenu()
{
    JFrame menu = new JFrame("Gym Membership");
    menu.setSize(800, 600);
    menu.setResizable(true);
    menu.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    menu.setVisible(true);
    menu.setLayout(null);
    menu.setExtendedState(JFrame.MAXIMIZED_BOTH);
    menu.getContentPane().setBackground(new Color(240, 248, 255));

    JLabel title = new JLabel("Select Member Category");
    title.setFont(new Font("Arial", Font.BOLD, 24));
    title.setBounds(500, 30, 400, 40);

    JButton regBtn = new JButton("Regular Member");
    regBtn.setBounds(520, 120, 240, 60);
    regBtn.addActionListener(e -> new RegularMemberGUI());
    regBtn.setLayout(new BorderLayout());

    JButton premBtn = new JButton("Premium Member");
    premBtn.setBounds(520, 210, 240, 60);
    premBtn.addActionListener(e -> new PremiumMemberGUI());
    premBtn.setLayout(new BorderLayout());
```

```
        menu.add(title);
        menu.add(regBtn);
        menu.add(premBtn);
        menu.setVisible(true);
    }
}
```

```
class RegularMemberGUI
```

```
{
    private JTextField id;
    private JTextField name;
    private JTextField phone;
    private JTextField email;
    private JTextField address;
    private JTextField referredby;
    private JTextField paidAmt;
    private JTextField removalreason;
    private JTextField dob;
    private JComboBox<String> gender;
    private JComboBox<String> method;
    private JComboBox<String> plans;
    private JTextArea displayArea;
    private JTextField id1;
    private JTextField id2;
    private JTextField id3;
    private JTextField id4;
```

```
RegularMemberGUI()
```

```
{
    JFrame win = new JFrame("Regular Member");
    win.setExtendedState(JFrame.MAXIMIZED_BOTH);
```

```
// Fill full screen
win.setLayout(null);
win.getContentPane().setBackground(new Color(240, 248, 255));

JLabel lbl = new JLabel("Regular Member Entry");
lbl.setBounds(500, 10, 300, 30);
lbl.setFont(new Font("Arial", Font.BOLD, 20));
win.add(lbl);

String[] genders = {"Select", "Male", "Female", "Other"};
String[] planList = {"Basic", "Regular", "Premium"};
String[] methods = {"Cash", "Card", "Online"};

id = createField("ID", 90, win);
name = createField("Name", 130, win);
phone = createField("Phone", 170, win);
email = createField("Email", 210, win);
address = createField("Address", 250, win);
referredby = createField("Referred By", 290, win);
paidAmt = createField("Paid Amount", 330, win);
removalreason = createField("Removal Reason", 370, win);
dob = createField("Date of Birth", 410, win);
// startDate field removed as requested

gender = new JComboBox<>(genders);
gender.setBounds(600, 90, 150, 25);
win.add(new JLabel("Gender") {{ setBounds(500, 90, 80, 25); }});
win.add(gender);

plans = new JComboBox<>(planList);
plans.setBounds(600, 130, 150, 25);
```

```
win.add(new JLabel("Plan") {{ setBounds(500, 130, 80, 25); }});
win.add(plans);

method = new JComboBox<>(methods);
method.setBounds(600, 170, 150, 25);
win.add(new JLabel("Pay Method") {{ setBounds(500, 170, 80, 25); }});
win.add(method);

// ID 1
JLabel idLabel1 = new JLabel("ID");
idLabel1.setBounds(500, 240, 100, 25);
win.add(idLabel1);

JTextField idField1 = new JTextField();
idField1.setBounds(600, 240, 150, 25);
win.add(idField1);

// ID 2
JLabel idLabel2 = new JLabel("ID");
idLabel2.setBounds(500, 280, 100, 25);
win.add(idLabel2);

JTextField idField2 = new JTextField();
idField2.setBounds(600, 280, 150, 25);
win.add(idField2);

// ID 3
JLabel idLabel3 = new JLabel("ID");
idLabel3.setBounds(500, 320, 100, 25);
win.add(idLabel3);
```

```
JTextField idField3 = new JTextField();  
idField3.setBounds(600, 320, 150, 25);  
win.add(idField3);
```

```
// ID 4
```

```
JLabel idLabel4 = new JLabel("ID");  
idLabel4.setBounds(500, 360, 100, 25);  
win.add(idLabel4);
```

```
JTextField idField4 = new JTextField();  
idField4.setBounds(600, 360, 150, 25);  
win.add(idField4);
```

```
JButton activate = new JButton("Activate");  
activate.setBounds(900, 80, 160, 30);  
win.add(activate);
```

```
JButton deactivate = new JButton("Deactivate");  
deactivate.setBounds(900, 140, 160, 30); // Next to activate  
win.add(deactivate);
```

```
JButton addRegular = new JButton("Add Regular Member");  
addRegular.setBounds(900, 200, 160, 30); // Next to deactivate  
win.add(addRegular);
```

```
JButton revertRegular = new JButton("Revert Regular Member");  
revertRegular.setBounds(875, 260, 210, 40); // Next to addRegular  
win.add(revertRegular);
```

```
activate.addActionListener(new ActionListener()
{
    public void actionPerformed(ActionEvent e)
    {
        // Example: Activate member by ID entered in 'id' text field
        String memberId = id.getText();
        if (memberId.isEmpty()) {
            JOptionPane.showMessageDialog(win, "Enter member ID to activate.");
        } else {
            // TODO: Add real activation logic here
            JOptionPane.showMessageDialog(win, "Member " + memberId + "
activated.");
        }
    }
});

deactivate.addActionListener(new ActionListener()
{
    public void actionPerformed(ActionEvent e)
    {
        // Example: Deactivate member by ID
        String memberId = id.getText();
        if (memberId.isEmpty())
        {
            JOptionPane.showMessageDialog(win, "Enter member ID to deactivate.");
        }
        else
        {
            // TODO: Add real deactivation logic here
```

```
JOptionPane.showMessageDialog(win, "Member " + memberId + "
deactivated.");
    }
}
});

addRegular.addActionListener(new ActionListener()
{
    public void actionPerformed(ActionEvent e)
    {
        // Example: Add regular member using entered data
        String memberName = name.getText();
        String memberId = id.getText();
        if (memberName.isEmpty() || memberId.isEmpty())
        {
            JOptionPane.showMessageDialog(win, "Enter member ID and name to
add.");
        }
        else
        {
            // TODO: Add real add member logic here
            JOptionPane.showMessageDialog(win, "Regular member " + memberName +
" added.");
        }
    }
});

revertRegular.addActionListener(new ActionListener()
{
    public void actionPerformed(ActionEvent e)
    {
```

```
// Example: Revert regular member by ID
String memberId = id.getText();
if (memberId.isEmpty())
{
    JOptionPane.showMessageDialog(win, "Enter member ID to revert.");
}
else
{
    // TODO: Add real revert logic here
    JOptionPane.showMessageDialog(win, "Member " + memberId + " reverted to
previous status.");
}
}
});
```

```
JButton mark = new JButton("Mark Attendance");
JButton upgrade = new JButton("Upgrade Plan");
JButton toPremium = new JButton("Change to Premium");
JButton save = new JButton("Save");
JButton clear = new JButton("Clear");
JButton display = new JButton("Display");
```

```
// Positioning: Top row
mark.setBounds(100, 530, 160, 30);    // Left
upgrade.setBounds(280, 530, 160, 30); // Center
toPremium.setBounds(460, 530, 180, 30); // Right
```

```
// Positioning: Second row
save.setBounds(660, 530, 160, 30);
```

```
display.setBounds(840, 530, 160, 30);

clear = new JButton("Clear");
clear.setBounds(1020, 530, 160, 30);
clear.addActionListener(new ActionListener()
{
    @Override
    public void actionPerformed(ActionEvent e)
    {
        id.setText("");
        name.setText("");
        phone.setText("");
        email.setText("");
        address.setText("");
        referredby.setText("");
        paidAmt.setText("");
        removalreason.setText("");
        dob.setText("");
        id1.setText("");
        id2.setText("");
        id3.setText("");
        id4.setText("");
    }
});

// Add to frame
win.add(mark);
win.add(upgrade);
win.add(toPremium);
win.add(save);
win.add(display);
```

```
win.add(clear);

mark.addActionListener(e -> JOptionPane.showMessageDialog(win, "Attendance
Marked: " + name.getText()));
upgrade.addActionListener(e -> JOptionPane.showMessageDialog(win, "Plan
Upgraded to: " + plans.getSelectedItem()));
toPremium.addActionListener(e -> JOptionPane.showMessageDialog(win,
"Switched to Premium Membership"));
save.addActionListener(e -> saveToFile());
clear.addActionListener(e -> clear());
display.addActionListener(e -> displayInfo());

win.setVisible(true);
}

private JTextField createField(String label, int y, JFrame frame)
{
    JLabel lbl = new JLabel(label);
    lbl.setBounds(50, y, 100, 25);
    JTextField tf = new JTextField();
    tf.setBounds(150, y, 200, 25);
    frame.add(lbl);
    frame.add(tf);
    return tf;
}

private void saveToFile()
{
    try (PrintWriter pw = new PrintWriter("regular.txt"))
    {
        pw.println(id.getText() + "," + name.getText());
    }
}
```

```
        JOptionPane.showMessageDialog(null, "Saved to File");
    }
    catch (IOException ex)
    {
        JOptionPane.showMessageDialog(null, "Error Saving");
    }
}

private void clear()
{
    id.setText("");
    name.setText("");
    phone.setText("");
    email.setText("");
    address.setText("");
    referredby.setText("");
    paidAmt.setText("");
    removalreason.setText("");
    dob.setText("");
    id1.setText("");
    id2.setText("");
    id3.setText("");
    id4.setText("");
    displayArea.setText("");
}

private void displayInfo()
{
    displayArea.setText("Name: " + name.getText() + "\nPhone: " + phone.getText());
}
```

```
}
```

```
class RegularMember
```

```
{
```

```
    private JFrame frame;  
    private JTextField idField;  
    private JTextField nameField;  
    private JTextField phoneField;  
    private JTextField emailField;  
    private JTextField addressField;  
    private JTextField referralField;  
    private JTextField dobField;  
    private JTextField startDateField;  
    private JTextField removalField;  
    private JComboBox<String> planBox;  
    private JComboBox<String> genderBox;  
    private JTextArea displayArea;
```

```
    public RegularMember()
```

```
    {
```

```
        frame = new JFrame("Regular Member");  
        frame.setSize(600, 600);  
        frame.setLayout(new BorderLayout()); // Change layout for better resizing  
  
        // Panel for form inputs  
        JPanel formPanel = new JPanel();  
        formPanel.setLayout(new GridBagLayout()); // GridBagLayout for better control  
  
        GridBagConstraints gbc = new GridBagConstraints();  
        gbc.insets = new Insets(5, 5, 5, 5);  
        gbc.fill = GridBagConstraints.HORIZONTAL;
```

```
gbc.anchor = GridBagConstraints.WEST;

// Row 0 - ID
gbc.gridx = 0; gbc.gridy = 0;
formPanel.add(new JLabel("ID:"), gbc);
idField = new JTextField(15);
gbc.gridx = 1;
formPanel.add(idField, gbc);

// Row 1 - Name
gbc.gridx = 0; gbc.gridy++;
formPanel.add(new JLabel("Name:"), gbc);
nameField = new JTextField(15);
gbc.gridx = 1;
formPanel.add(nameField, gbc);

// Row 2 - Phone
gbc.gridx = 0; gbc.gridy++;
formPanel.add(new JLabel("Phone:"), gbc);
phoneField = new JTextField(15);
gbc.gridx = 1;
formPanel.add(phoneField, gbc);

// Row 3 - Email
gbc.gridx = 0; gbc.gridy++;
formPanel.add(new JLabel("Email:"), gbc);
emailField = new JTextField(15);
gbc.gridx = 1;
formPanel.add(emailField, gbc);

// Row 4 - Address
```

```
gbc.gridx = 0; gbc.gridy++;
formPanel.add(new JLabel("Address:"), gbc);
addressField = new JTextField(15);
gbc.gridx = 1;
formPanel.add(addressField, gbc);

// Row 5 - Referred By
gbc.gridx = 0; gbc.gridy++;
formPanel.add(new JLabel("Referred By:"), gbc);
referralField = new JTextField(15);
gbc.gridx = 1;
formPanel.add(referralField, gbc);

// Row 6 - Plan
gbc.gridx = 0; gbc.gridy++;
formPanel.add(new JLabel("Plan:"), gbc);
planBox = new JComboBox<>(new String[]{"Basic", "Regular", "Premium"});
gbc.gridx = 1;
formPanel.add(planBox, gbc);

// Removed paidField here as requested (was below planBox)

// Row 7 - Gender
gbc.gridx = 0; gbc.gridy++;
formPanel.add(new JLabel("Gender:"), gbc);
genderBox = new JComboBox<>(new String[]{"Male", "Female", "Other"});
gbc.gridx = 1;
formPanel.add(genderBox, gbc);

// Row 9 - DOB
gbc.gridx = 0; gbc.gridy++;
```

```
formPanel.add(new JLabel("Date of Birth:"), gbc);
dobField = new JTextField(10);
gbc.gridx = 1;
formPanel.add(dobField, gbc);

// Row 10 - Start Date
gbc.gridx = 0; gbc.gridy++;
formPanel.add(new JLabel("Start Date:"), gbc);
startDateField = new JTextField(10);
gbc.gridx = 1;
formPanel.add(startDateField, gbc);

// Row 11 - Removal Reason
gbc.gridx = 0; gbc.gridy++;
formPanel.add(new JLabel("Removal Reason:"), gbc);
removalField = new JTextField(15);
gbc.gridx = 1;
formPanel.add(removalField, gbc);

frame.add(formPanel, BorderLayout.NORTH);

// Text area for display info
displayArea = new JTextArea(8, 50);
displayArea.setEditable(false);
JScrollPane scrollPane = new JScrollPane(displayArea);
frame.add(scrollPane, BorderLayout.CENTER);

// Panel for buttons
JPanel buttonPanel = new JPanel();
buttonPanel.setLayout(new GridLayout(1, 6, 10, 10)); // 6 buttons in 1 row with
gaps
```

```

JButton saveButton = new JButton("Save to File");
JButton readButton = new JButton("Read from File");
JButton displayButton = new JButton("Display");
JButton upgradeButton = new JButton("Upgrade Plan");
JButton markButton = new JButton("Mark Attendance");
JButton changeToPremium = new JButton("Switch to Premium");

// All buttons added to button panel (GridLayout makes them equal size)
buttonPanel.add(saveButton);
buttonPanel.add(readButton);
buttonPanel.add(displayButton);
buttonPanel.add(upgradeButton);
buttonPanel.add(markButton);
buttonPanel.add(changeToPremium);

frame.add(buttonPanel, BorderLayout.SOUTH);

// Button actions
saveButton.addActionListener(e -> saveToFile());
readButton.addActionListener(e -> readFromFile());
displayButton.addActionListener(e -> display());
markButton.addActionListener(e -> JOptionPane.showMessageDialog(frame,
"Attendance marked.));
    upgradeButton.addActionListener(e ->
planBox.setSelectedIndex((planBox.getSelectedIndex() + 1) % 3));
    changeToPremium.addActionListener(e ->
    {
        frame.dispose();
        new PremiumMemberGUI();
    });
});
```

```
frame.setExtendedState(JFrame.MAXIMIZED_BOTH); // Start maximized
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setVisible(true);
}
```

```
private void saveToFile()
{
    try (PrintWriter writer = new PrintWriter("regular_member.txt"))
    {
        writer.println("ID: " + idField.getText());
        writer.println("Name: " + nameField.getText());
        writer.println("Phone: " + phoneField.getText());
        writer.println("Email: " + emailField.getText());
        writer.println("Address: " + addressField.getText());
        writer.println("Referral: " + referralField.getText());
        writer.println("Plan: " + planBox.getSelectedItem());
        // paidField removed, so skip
        writer.println("Gender: " + genderBox.getSelectedItem());
        writer.println("DOB: " + dobField.getText());
        writer.println("Start Date: " + startDateField.getText());
        writer.println("Removal: " + removalField.getText());
        JOptionPane.showMessageDialog(frame, "Saved Successfully.");
    }
    catch (IOException ex)
    {
        JOptionPane.showMessageDialog(frame, "Error Saving File.");
    }
}
```

```
private void readFromFile()
```

```
{
    try (BufferedReader reader = new BufferedReader(new
FileReader("regular_member.txt")))
    {
        displayArea.setText("");
        String line;
        while ((line = reader.readLine()) != null)
        {
            displayArea.append(line + "\n");
        }
    }
    catch (IOException ex)
    {
        JOptionPane.showMessageDialog(frame, "Error Reading File.");
    }
}

private void display()
{
    displayArea.setText(
        "ID: " + idField.getText() + "\n" +
        "Name: " + nameField.getText() + "\n" +
        "Phone: " + phoneField.getText() + "\n" +
        "Email: " + emailField.getText() + "\n" +
        "Address: " + addressField.getText() + "\n" +
        "Referred By: " + referralField.getText() + "\n" +
        "Plan: " + planBox.getSelectedItem() + "\n" +
        //"Paid: " + paidField.getText() + "\n" + // removed paidField display
        "Gender: " + genderBox.getSelectedItem() + "\n" +
        "DOB: " + dobField.getText() + "\n" +
        "Start Date: " + startDateField.getText() + "\n" +
```

```
        "Removal Reason: " + removalField.getText()
    );
}

// For testing
public static void main(String[] args)
{
    SwingUtilities.invokeLater(() -> new RegularMember());
}
}
```

```
class PremiumMemberGUI
{
    private JTextField id;
    private JTextField name;
    private JTextField phone;
    private JTextField email;
    private JTextField address;
    private JTextField trainer;
    private JTextField payment;
    private JTextField removalreason;
    private JTextField dob;
    private JComboBox<String> gender;
    private JComboBox<String> method;
    private JComboBox<String> plans;
    private JTextArea view;
    private JTextArea displayArea;
    private JTextField id1;
    private JTextField id2;
    private JTextField id3;
```

```
private JTextField id4;
private JTextField referredby;

public PremiumMemberGUI()
{
    JFrame win = new JFrame("Premium Member");
    win.setExtendedState(JFrame.MAXIMIZED_BOTH);
    win.setLayout(null);
    win.getContentPane().setBackground(new Color(240, 248, 255));

    JLabel lbl = new JLabel("Premium Member Entry");
    lbl.setBounds(500, 10, 300, 30);
    lbl.setFont(new Font("Arial", Font.BOLD, 20));
    win.add(lbl);

    String[] genders = {"Select", "Male", "Female", "Other"};
    String[] methods = {"Cash", "Card", "Online"};
    String[] planList = {"Basic", "Regular", "Premium"};

    id = createField("ID", 50, win);
    name = createField("Name", 90, win);
    phone = createField("Phone", 130, win);
    email = createField("Email", 170, win);
    address = createField("Address", 210, win);
    trainer = createField("Trainer", 250, win);
    payment = createField("Amount", 290, win);
    removalreason = createField("Removal", 330, win);
    dob = createField("DOB", 370, win);
    // startDate removed as per request
```

```
gender = new JComboBox<>(genders);
gender.setBounds(600, 90, 150, 25);
win.add(new JLabel("Gender") {{ setBounds(500, 90, 80, 25); }});
win.add(gender);

method = new JComboBox<>(methods);
method.setBounds(600, 130, 150, 25);
win.add(new JLabel("Pay Method") {{ setBounds(500, 130, 100, 25); }});
win.add(method);

plans = new JComboBox<>(planList);
plans.setBounds(600, 170, 150, 25);
win.add(new JLabel("Plan") {{ setBounds(500, 170, 120, 25); }});
win.add(plans);

// ID 1
JLabel idLabel1 = new JLabel("ID");
idLabel1.setBounds(500, 240, 100, 25);
win.add(idLabel1);

JTextField idField1 = new JTextField();
idField1.setBounds(600, 240, 150, 25);
win.add(idField1);

// ID 2
JLabel idLabel2 = new JLabel("ID");
idLabel2.setBounds(500, 280, 100, 25);
win.add(idLabel2);

JTextField idField2 = new JTextField();
idField2.setBounds(600, 280, 150, 25);
```

```
win.add(idField2);

// ID 3
JLabel idLabel3 = new JLabel("ID");
idLabel3.setBounds(500, 320, 100, 25);
win.add(idLabel3);

JTextField idField3 = new JTextField();
idField3.setBounds(600, 320, 150, 25);
win.add(idField3);

// ID 4
JLabel idLabel4 = new JLabel("ID");
idLabel4.setBounds(500, 360, 100, 25);
win.add(idLabel4);

JTextField idField4 = new JTextField();
idField4.setBounds(600, 360, 150, 25);
win.add(idField4);

JButton activate = new JButton("Activate");
activate.setBounds(900, 80, 160, 30);
win.add(activate);

JButton deactivate = new JButton("Deactivate");
deactivate.setBounds(900, 140, 160, 30);
win.add(deactivate);

JButton addPremium = new JButton("Add Premium Member");
addPremium.setBounds(900, 260, 210, 40);
```

```
win.add(addPremium);
```

```
JButton revertPremium = new JButton("Revert Premium Member");
```

```
revertPremium.setBounds(900, 260, 210, 40);
```

```
win.add(revertPremium);
```

```
activate.addActionListener(new ActionListener()
```

```
{
```

```
    public void actionPerformed(ActionEvent e)
```

```
    {
```

```
        // Example: Activate member by ID entered in 'id' text field
```

```
        String memberId = id.getText();
```

```
        if (memberId.isEmpty()) {
```

```
            JOptionPane.showMessageDialog(win, "Enter member ID to activate.");
```

```
        } else {
```

```
            // TODO: Add real activation logic here
```

```
            JOptionPane.showMessageDialog(win, "Member " + memberId + "
```

```
activated.");
```

```
        }
```

```
    }
```

```
});
```

```
deactivate.addActionListener(new ActionListener()
```

```
{
```

```
    public void actionPerformed(ActionEvent e)
```

```
    {
```

```
        // Example: Deactivate member by ID
```

```
        String memberId = id.getText();
```

```
        if (memberId.isEmpty())
```

```
        {
```

```
        JOptionPane.showMessageDialog(win, "Enter member ID to deactivate.");
    }
    else
    {
        // TODO: Add real deactivation logic here
        JOptionPane.showMessageDialog(win, "Member " + memberId + "
deactivated.");
    }
}
});

addPremium.addActionListener(new ActionListener()
{
    public void actionPerformed(ActionEvent e)
    {
        // Add premium member using entered data
        String memberName = name.getText();
        String memberId = id.getText();
        if (memberName.isEmpty() || memberId.isEmpty())
        {
            JOptionPane.showMessageDialog(win, "Enter member ID and name to
add.");
        }
        else
        {
            // Add real add member logic here
            JOptionPane.showMessageDialog(win, "Premium member " + memberName
+ " added.");
        }
    }
});
```

```
revertPremium.addActionListener(new ActionListener()
{
    public void actionPerformed(ActionEvent e)
    {
        // Example: Revert regular member by ID
        String memberId = id.getText();
        if (memberId.isEmpty())
        {
            JOptionPane.showMessageDialog(win, "Enter member ID to revert.");
        }
        else
        {
            // TODO: Add real revert logic here
            JOptionPane.showMessageDialog(win, "Member " + memberId + " reverted to
previous status.");
        }
    }
});
```

```
JButton mark = new JButton("Mark Attendance");
JButton upgrade = new JButton("Upgrade Plan");
JButton toRegular = new JButton("Change to Regular");
JButton save = new JButton("Save");
JButton clear = new JButton("Clear");
JButton display = new JButton("Display");
```

```
// Positioning: Top row
mark.setBounds(100, 530, 160, 30);    // Left
```

```
upgrade.setBounds(280, 530, 160, 30); // Center
toRegular.setBounds(460, 530, 180, 30); // Right

// Positioning: Second row
save.setBounds(660, 530, 160, 30);
display.setBounds(840, 530, 160, 30);

clear = new JButton("Clear");
clear.setBounds(1020, 530, 160, 30);
clear.addActionListener(new ActionListener()
{
    @Override
    public void actionPerformed(ActionEvent e)
    {
        id.setText("");
        name.setText("");
        phone.setText("");
        email.setText("");
        address.setText("");
        trainer.setText("");
        removalreason.setText("");
        dob.setText("");
        id1.setText("");
        id2.setText("");
        id3.setText("");
        id4.setText("");
    }
});

// Add to frame
win.add(mark);
```

```
win.add(upgrade);
win.add(toRegular);
win.add(save);
win.add(display);
win.add(clear);

mark.addActionListener(e -> JOptionPane.showMessageDialog(win, "Attendance
Mrked: " + name.getText()));
toRegular.addActionListener(e -> JOptionPane.showMessageDialog(win,
"Switched to Regular Membership"));
save.addActionListener(e -> saveToFile());
clear.addActionListener(e -> clear());
display.addActionListener(e -> display());

win.setVisible(true);
}

private JTextField createField(String label, int y, JFrame frame)
{
    JLabel lbl = new JLabel(label);
    lbl.setBounds(50, y, 100, 25);
    JTextField tf = new JTextField();
    tf.setBounds(150, y, 200, 25);
    frame.add(lbl);
    frame.add(tf);
    return tf;
}

private void saveToFile()
{
    try (PrintWriter pw = new PrintWriter("premium.txt"))
```

```
{
    pw.println(id.getText() + "," + name.getText());
    JOptionPane.showMessageDialog(null, "Saved to File");
}
catch (IOException ex)
{
    JOptionPane.showMessageDialog(null, "Save Failed");
}
}

private void clear()
{
    id.setText("");
    name.setText("");
    phone.setText("");
    email.setText("");
    address.setText("");
    trainer.setText("");
    removalreason.setText("");
    dob.setText("");
    id1.setText("");
    id2.setText("");
    id3.setText("");
    id4.setText("");
    displayArea.setText("");
}

private void display()
{
    view.setText("Name: " + name.getText() + "\nPhone: " + phone.getText());
```

```
    }  
}
```

```
class PremiumMember
```

```
{  
    private JFrame frame;  
    private JTextField idField;  
    private JTextField nameField;  
    private JTextField phoneField;  
    private JTextField emailField;  
    private JTextField addressField;  
    private JTextField loyaltyField;  
    private JTextField paidField;  
    private JTextField dobField;  
    private JTextField startDateField;  
    private JTextField removalField;  
    private JComboBox<String> planBox;  
    private JComboBox<String> genderBox;  
    private JTextArea displayArea;  
  
    public PremiumMember()  
    {  
        frame = new JFrame("Premium Member");  
        frame.setExtendedState(JFrame.MAXIMIZED_BOTH); // Maximize window on  
open  
        frame.setLayout(new FlowLayout());  
  
        frame.add(new JLabel("ID:"));  
        idField = new JTextField(15);  
        frame.add(idField);  
    }  
}
```

```
frame.add(new JLabel("Name:"));
nameField = new JTextField(15);
frame.add(nameField);

frame.add(new JLabel("Phone:"));
phoneField = new JTextField(15);
frame.add(phoneField);

frame.add(new JLabel("Email:"));
emailField = new JTextField(15);
frame.add(emailField);

frame.add(new JLabel("Address:"));
addressField = new JTextField(15);
frame.add(addressField);

frame.add(new JLabel("Loyalty Points:"));
loyaltyField = new JTextField(10);
frame.add(loyaltyField);

frame.add(new JLabel("Plan:"));
planBox = new JComboBox<>(new String[]{"Basic", "Regular", "Premium"});
frame.add(planBox);

// Removed Paid Amount field below plan as requested
// frame.add(new JLabel("Paid Amount:"));
// paidField = new JTextField(10);
// frame.add(paidField);

frame.add(new JLabel("Gender:"));
genderBox = new JComboBox<>(new String[]{"Male", "Female", "Other"});
```

```
frame.add(genderBox);

frame.add(new JLabel("Date of Birth:"));
dobField = new JTextField(10);
frame.add(dobField);

frame.add(new JLabel("Start Date:"));
startDateField = new JTextField(10);
frame.add(startDateField);

frame.add(new JLabel("Removal Reason:"));
removalField = new JTextField(15);
frame.add(removalField);

JButton saveButton = new JButton("Save to File");
JButton readButton = new JButton("Read from File");
JButton displayButton = new JButton("Display Info");
JButton downgradeButton = new JButton("Downgrade Plan");
JButton markButton = new JButton("Mark Attendance");
JButton switchToRegular = new JButton("Switch to Regular");

// Set preferred sizes for buttons to fit the text nicely
Dimension btnSize = new Dimension(150, 30);
saveButton.setPreferredSize(btnSize);
readButton.setPreferredSize(btnSize);
displayButton.setPreferredSize(btnSize);
downgradeButton.setPreferredSize(btnSize);
markButton.setPreferredSize(btnSize);
switchToRegular.setPreferredSize(btnSize);

displayArea = new JTextArea(8, 50);
```

```
frame.add(new JScrollPane(displayArea));

frame.add(saveButton);
frame.add(readButton);
frame.add(displayButton);
frame.add(downgradeButton);
frame.add(markButton);
frame.add(switchToRegular);

saveButton.addActionListener(e -> saveToFile());
readButton.addActionListener(e -> readFromFile());
displayButton.addActionListener(e -> display());
markButton.addActionListener(e -> JOptionPane.showMessageDialog(frame,
"Attendance marked."));
downgradeButton.addActionListener(e ->
planBox.setSelectedIndex((planBox.getSelectedIndex() + 1) % 3));
switchToRegular.addActionListener(e ->
{
    frame.dispose();
    new RegularMemberGUI();
});

frame.setVisible(true);
}

private void saveToFile()
{
    try (PrintWriter writer = new PrintWriter("premium_member.txt"))
    {
        writer.println("ID: " + idField.getText());
        writer.println("Name: " + nameField.getText());
    }
}
```

```
writer.println("Phone: " + phoneField.getText());
writer.println("Email: " + emailField.getText());
writer.println("Address: " + addressField.getText());
writer.println("Loyalty Points: " + loyaltyField.getText());
writer.println("Plan: " + planBox.getSelectedItem());
// Removed Paid field line because field removed
// writer.println("Paid: " + paidField.getText());
writer.println("Gender: " + genderBox.getSelectedItem());
writer.println("DOB: " + dobField.getText());
writer.println("Start Date: " + startDateField.getText());
writer.println("Removal: " + removalField.getText());
JOptionPane.showMessageDialog(frame, "Saved Successfully.");
}
catch (IOException ex)
{
    JOptionPane.showMessageDialog(frame, "Error Saving File.");
}
}

private void readFromFile()
{
    try (BufferedReader reader = new BufferedReader(new
FileReader("premium_member.txt")))
    {
        displayArea.setText("");
        String line;
        while ((line = reader.readLine()) != null)
        {
            displayArea.append(line + "\n");
        }
    }
}
```

```
        catch (IOException ex)
        {
            JOptionPane.showMessageDialog(frame, "Error Reading File.");
        }
    }

    private void display()
    {
        displayArea.setText
        (
            "ID: " + idField.getText() + "\n" +
            "Name: " + nameField.getText() + "\n" +
            "Phone: " + phoneField.getText() + "\n" +
            "Email: " + emailField.getText() + "\n" +
            "Address: " + addressField.getText() + "\n" +
            "Loyalty Points: " + loyaltyField.getText() + "\n" +
            "Plan: " + planBox.getSelectedItem() + "\n" +
            // Removed Paid from displayInfo as well
            // "Paid: " + paidField.getText() + "\n" +
            "Gender: " + genderBox.getSelectedItem() + "\n" +
            "DOB: " + dobField.getText() + "\n" +
            "Start Date: " + startDateField.getText() + "\n" +
            "Removal Reason: " + removalField.getText()
        );
    }
}
```